

Project - High Level Design

on

Autonomous Retail Research Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Kratin Verma	EN22CS301517
2	Karishma Patidar	EN22CS301484
3	Kanishka Patidar	EN22CS301480
4	Kratika Tomar	EN22CS301516
5	Jay Yadav	EN22CS301462

Group Name: Group 01D5

Project Number: AAI-01

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Maya Yadav Baniya

Academic Year: 2025 -2026

1.Introduction.

The Autonomous Retail Researcher Agent is an AI-powered system designed to automate retail market research. It uses Large Language Models (LLMs), agent frameworks, and web tools to collect, analyze, and summarize retail data such as trends, competitors, pricing, and customer sentiment.

The system reduces manual effort and provides structured insights for business decision-making.

1.1 Scope of the document.

This document describes the High-Level Design (HLD) of the Autonomous Retail Researcher Agent implemented using FastAPI, Streamlit, Gemini 2.5 Flash, Tavily Web Search, MongoDB, and Sentence Transformers.

The system provides:

- Conversational retail research
- Web-augmented AI intelligence
- Structured executive reports
- Semantic memory search
- Persistent storage
- KPI dashboard analytics

This document focuses on architectural structure, system behavior, data design, and operational characteristics aligned strictly with the implemented code.

1.2 Intended Audience

- System Architects
- Backend Developers
- Frontend Developers
- AI Engineers
- Academic Evaluators
- DevOps Engineers

1.3 System overview.

The Autonomous Retail Researcher Agent is an AI-powered conversational system that:

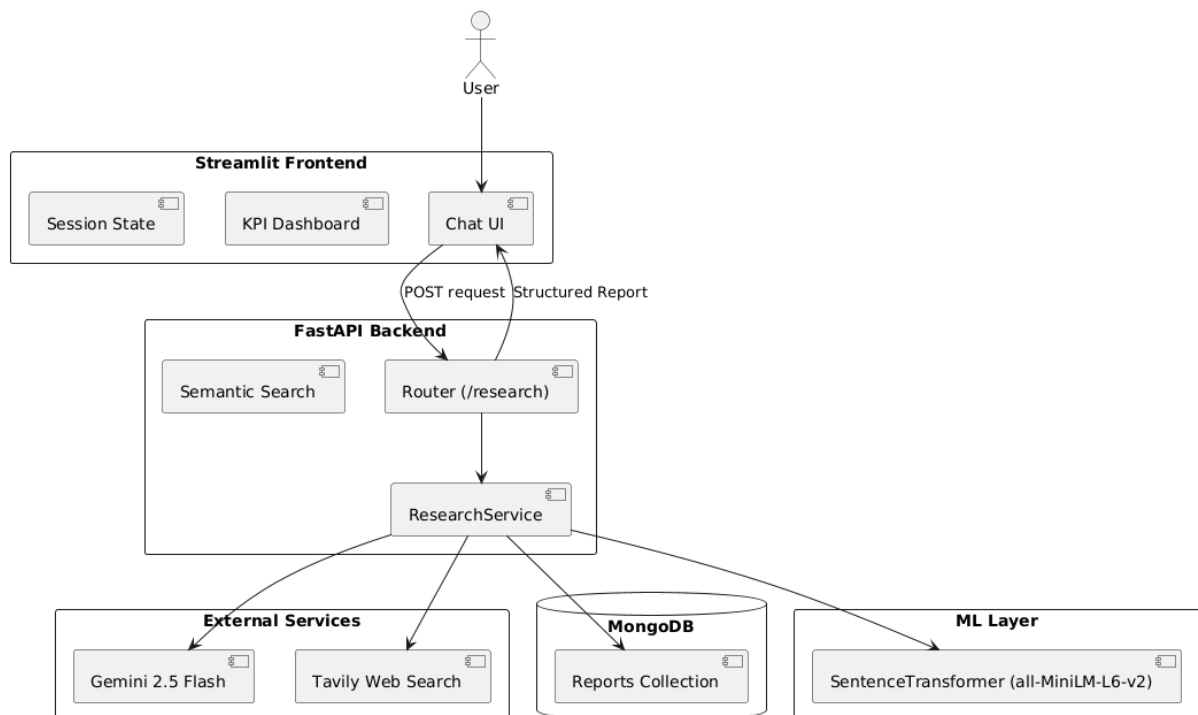
1. Accepts retail research queries.
2. Performs real-time web search using Tavily.
3. Generates executive-level structured reports using Gemini 2.5 Flash.
4. Stores reports with embeddings in MongoDB.
5. Enables semantic similarity search.

6. Displays structured research reports in a Streamlit dashboard.
- The system follows a layered microservice architecture with external AI.

2. System Design.

2.1 Application Design

1. Presentation Layer → Streamlit UI
2. API Layer → FastAPI Router
3. Service Layer → ResearchService
4. AI Layer → Gemini 2.5 Flash
5. Search Layer → Tavily API
6. Data Layer → MongoDB
7. Intelligence Layer → Sentence Transformer Embeddings

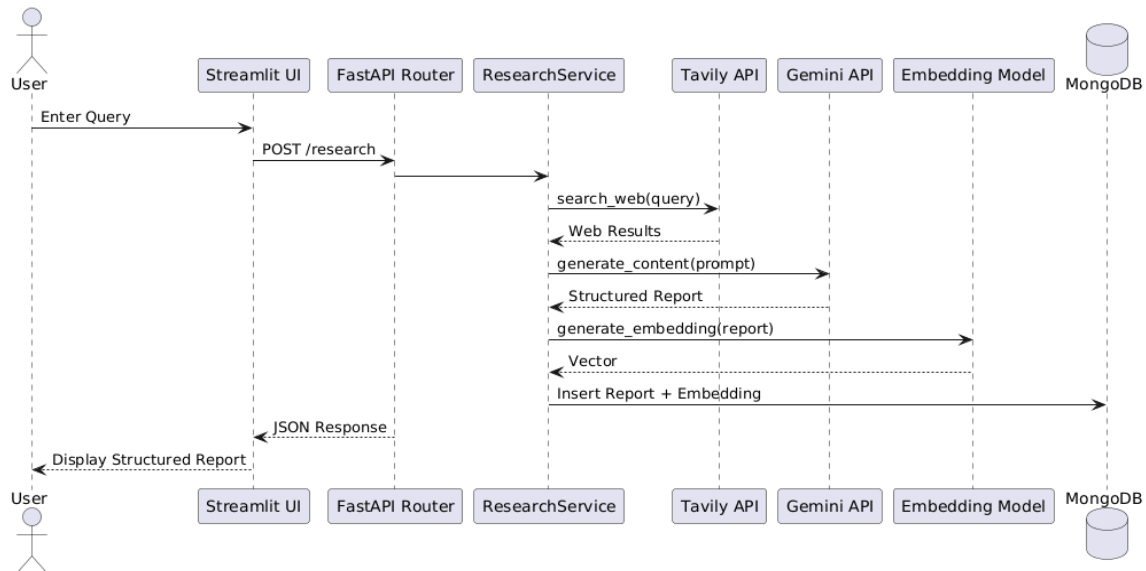


2.2 Process Flow.

1. User submits query in Streamlit.
2. Frontend sends POST /research with:
 - query
 - chat_history
3. FastAPI router validates request.
4. ResearchService:
 - Builds conversation context
 - Calls search_web() (Tavily)
 - Builds structured prompt

- Calls generate_report() (Gemini)
- Generates embedding
- Stores document in MongoDB

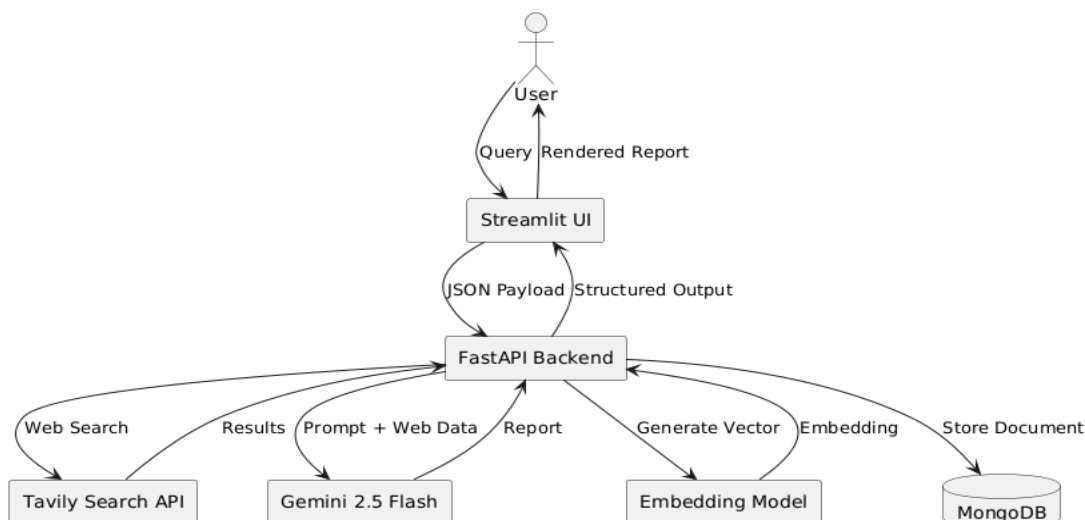
5. Report returned to frontend.
6. Streamlit formats structured report.



2.3 Information Flow.

There are four major data flows:

1. Query Flow
User → Streamlit → FastAPI
2. Web Intelligence Flow
FastAPI → Tavily → FastAPI
3. AI Generation Flow
FastAPI → Gemini → FastAPI
4. Persistence Flow
FastAPI → MongoDB



2.4 Components Design

1 Streamlit Frontend

- Chat Interface
- Session State Management
- Sidebar History Loader
- KPI Metrics Generator
- Structured Report Formatter

2 FastAPI Backend

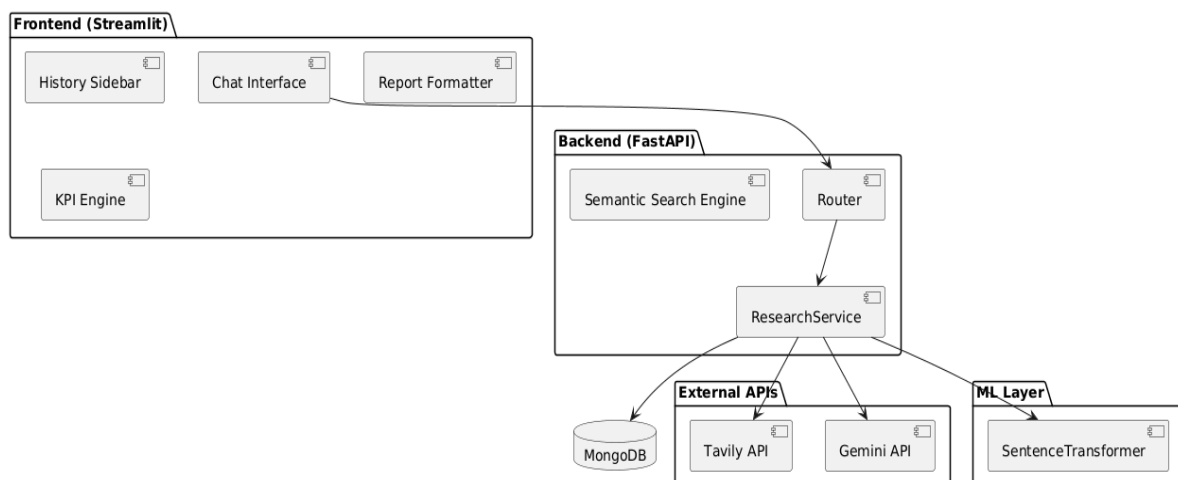
- /research
- /search-history
- /report/{id}
- Error Handling
- JSON Serialization

3 ResearchService

- Context builder
- Tavily integration
- Gemini integration
- Embedding generation
- MongoDB insertion
- Semantic search

4 Embedding Engine

- SentenceTransformer (all-MiniLM-L6-v2)
- Vector generation
- Cosine similarity search



2.5 Key Design Considerations

- Context continuity via chat history
- Modular separation of concerns
- Structured report formatting
- Stateless backend architecture
- Scalability readiness
- Error isolation

2.6 API Catalogue.

POST /research	Generates structured research report.
GET /search-history	Returns last 50 reports.
GET /report/{id}	Returns full report + chat history.

3. Data Design.

3.1 Data Model:-

```
{
  "_id": ObjectId,
  "query": "Retail AI Trends",
  "report": "Executive Summary: ...",
  "chat_history": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."}
  ],
  "sources": [
    {"title": "...", "url": "..."}
  ],
  "embedding": [0.124, 0.884, ...],
  "timestamp": ISODate
}
```

3.2 Data Access Mechanism

- PyMongo driver
- Indexed timestamp field
- Sorted retrieval
- ID-based lookup

3.3 Data Retention Policies

- Default: Permanent storage
- Future: Auto-archival after X months
- Backup via MongoDB Atlas

3.4 Data Migration

- JSON export
- Cloud migration support
- Replica set scaling

4. Interfaces

Interface	Type
User → Frontend	Web UI
Frontend → Backend	REST API
Backend → Gemini	HTTPS API
Backend → MongoDB	Driver

5. State and Session Management

Frontend:

- Uses `st.session_state`
- Maintains live conversation

Backend:

- Stateless
- Relies on MongoDB persistence

Conversation continuity:

- `chat_history` passed in every request

6. Caching

Current: No caching

Future:

- Redis caching
- Query similarity detection
- LLM response reuse

7. Non-Functional Requirements

7.1 Security Aspects

- Environment variable management (.env)
- Secure MongoDB connection (TLS)
- API key protection
- Input validation
- Exception handling
- HTTPS external calls

7.2 Performance Aspects

- Average response: 3–6 seconds
- Indexed DB retrieval
- Embedding similarity threshold: 0.70
- Horizontal backend scaling possible
- Microservice ready architecture

8. References

- FastAPI Documentation
- Streamlit Documentation
- MongoDB Atlas Documentation
- Gemini API Documentation
- REST API Design Standards